STUDY GUIDE



PYEBOOK

AP®
CSA

ALL RIGHTS RESERVED BY PYEBOOK © 2021

Table of Contents

1. What is a class?
2. What is an object/instance of a class?
3. Methods in a class
4. Scope of the variables used within a class and within a method
5. Constructors in a class
6. Method and Constructor Overloading
7. this keyword
8. Static members and methods
9. Pass by value & Pass by reference
10. Working with references & aliasing
11. Math class
12. String class

1. What Is a Class?

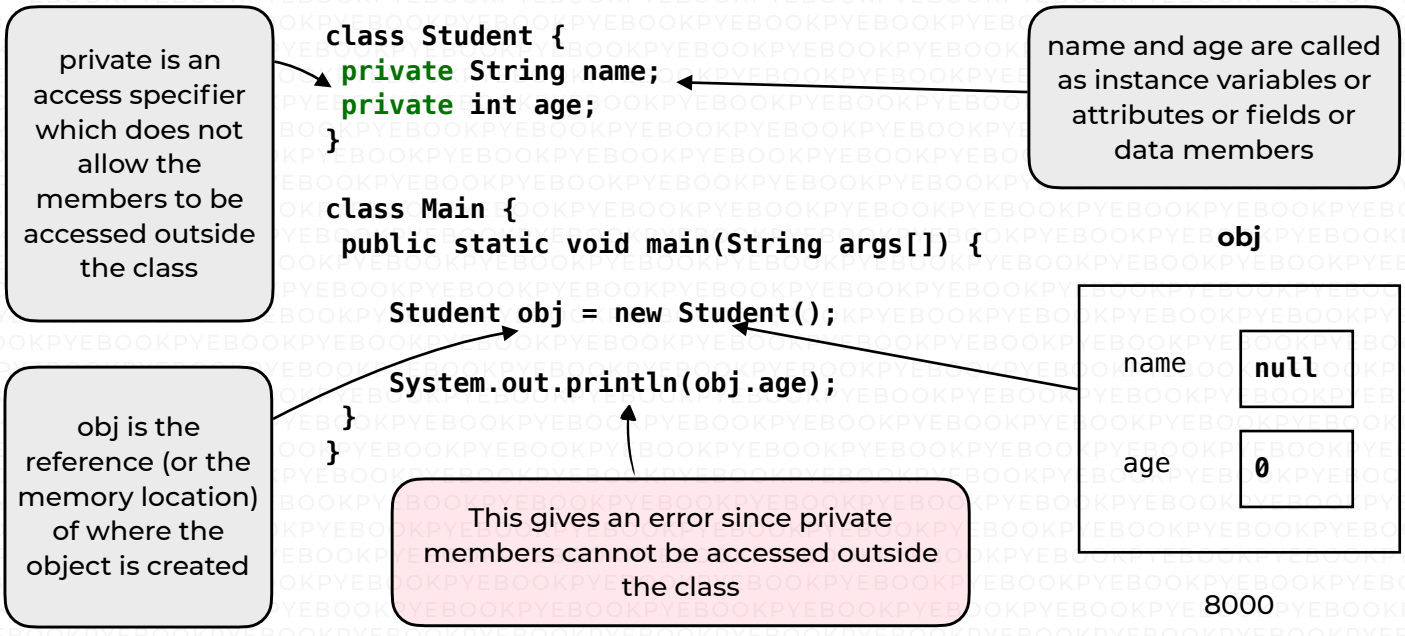
CONCEPT

`Class` is a logical name given to a collection of attributes usually of different data types. A `class` is also called a secondary data type or a user-defined data type.

2. What Is an Object/Instance of a Class?

CONCEPT

An object is the actual manifestation of the class. The class defines the blueprint whereas the object is the actual creation which occupies memory as per the design specified by the class.



3. Methods in a Class

CONCEPT & EXAMPLE

A method is a set of statements that performs a specific task. Usually, they are used to manipulate certain data members of the class. Unlike instance variables, methods are usually kept `public`.

General structure of a **method definition** in a class:

```
<Access Specifier> <Return Type> <Method Name> (<List of parameters separated by comma>) {
  <Body of the method>
}
```

General structure of a **method call** outside the class:

```
<Instance of the class>.<method-name>(<List of parameters>)
```

What does the `return type` mean?
Return type is the data type of the answer returned by the method. If the method doesn't return any value, then the return type is `void`.

What do you mean by parameters?
Parameters are variables (apart from the instance variables) needed before the function starts executing its work.

A. Return Type — void | Parameters — none

When the return type of the method is `void`, this means the method will not return any value.

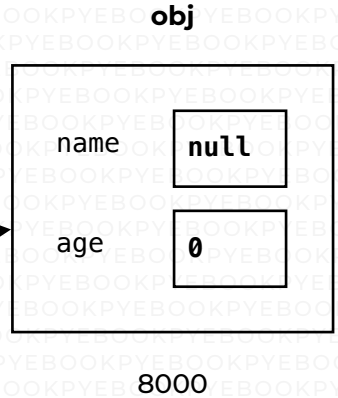
`public` is an access specifier which allows members to be accessible anywhere

```
class Student {
    private String name;
    private int age;

    public void showAge() {
        System.out.print(age);
    }
}

class Main {
    public static void main(String args[]) {
        Student obj = new Student();
        obj.showAge();
    }
}
```

Will output `0`



B. Return Type — void | Parameters — present

```
class Student {

    private String name;
    private int age;

    public void setAge(int val){
        age = val;
    }
}

class Main {
    public static void main(String args[]) {

        Student obj = new Student();
        obj.setAge(15);
    }
}
```

val: 15

obj: name = null, age = 0

8000

After setAge is called

obj: name = null, age = 15

8000

setAge is **called as a mutator** since it sets the value of a member

C. Return Type — present | Parameters — present

```
class Student {

    private String name;
    private int age;

    public void setAge(int val) {
        age = val;
    }

    public int getIncreasedAge() {
        int val = age + 10;
        return val;
    }
}
class Main {
    public static void main(String args[]) {

        Student obj = new Student();
        obj.setAge(15);

        int value = obj.getIncreasedAge();
    }
}
```

If a method has a return type, then it must return a value of that type **always**.

Here getIncreasedAge is also called as an accessor since it access a member of the class.

In its simplest form, an accessor just returns the value of a member

val: 25

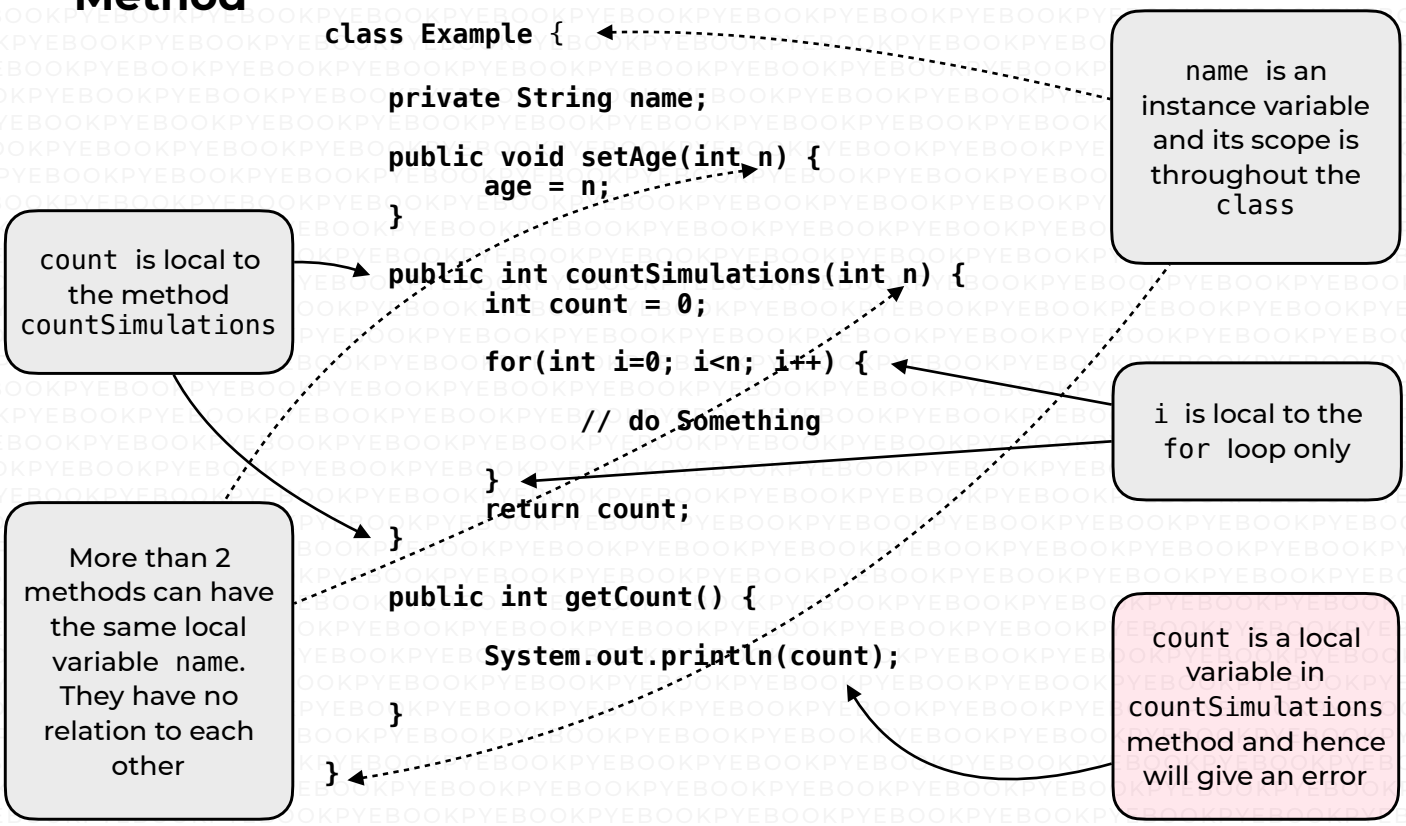
The state of the instance after the setAge is called.

obj: name = null, age = 15

8000

value will have 25 which is the number returned by getIncreasedAge

4. Scope of the Variables Used Within a Class and Within a Method



5. Constructors in a Class

CONCEPT

1. The purpose of a constructor is to create the instance of the class and assign initial values to the instance members.
2. It has the same name as the class and does not have a return type.
3. While creating the object, the constructor always needs to be called with the new keyword.
4. For instance members of a primitive type, the default values are 0 for int, 0.0 for double and false for boolean. Whereas, secondary data types are initialised with the value null.
5. You can create multiple constructors in the same class. They need to differ with respect to the parameters.

EXAMPLE

```
class Student {

    private String name;
    private int age;

    public Student() {
        name = null;
        age = 0;
    }

    public Student() {
        name = "AB";
        age = 20;
    }

    public Student(String n, int a) {
        name = n;
        age = a;
    }
}

class Main {
    public static void main(String args[]) {

        Student obj = new Student();

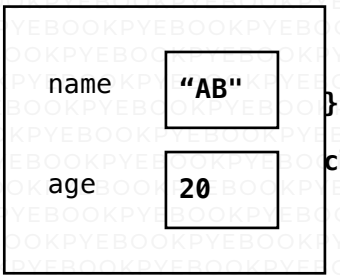
        Student param = new Student("XY",12);
    }
}
```

This is the default constructor Java includes in the class if you do not add one

This default Constructor has no parameters.

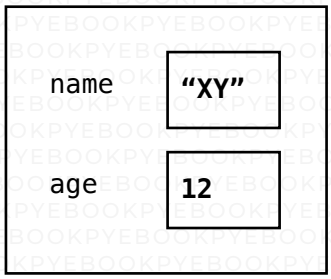
This is constructor with parameters. You can have as many constructors as needed as long as they differ in parameters

obj



8000

param



8000

Object creation using default constructor

Object creation using the parameterised constructor

6. Method and Constructor Overloading

CONCEPT

Constructors and methods can be overloaded. **Overloading** is the process of having multiple methods or constructors with the **same name but different parameters**. Difference in parameters can be made by:

1. Number of parameters
2. Data type of parameters
3. Order in which parameters are declared

EXAMPLE

Consider the `class Student` as seen below:

```
class Student {

    public void setInfo(int value) {
        // Does something
    }

    public void setInfo(String val) {
        // Does something
    }

    public void setInfo(int v, double r) {
        // Does something
    }
}
```

For the above class, here is a list of method declarations that will be allowed and that won't be:

Method	After Compiling	Reason
`public int setInfo(String n)`	Error	Return type being different than the existing methods in the class DOES NOT make it eligible for overloading.
`public void setInfo(String na, int ag)`	No Error	The number of parameters are different than the one in the class.
`public void setInfo(double na, int ag)`	No Error	The order of the parameters is different
`public void setInfo(int n)`	Error	There is already a `setInfo` method which takes an int as a parameter. The name of the parameter being different is

7. this Keyword

CONCEPT

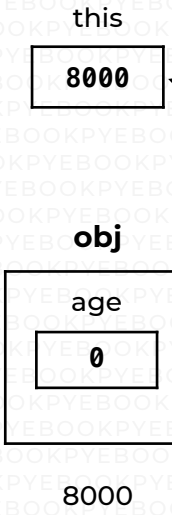
The keyword `this` is **present automatically inside non-static methods** and refers to the **reference of the calling object.**

```
class Student {
    private int age;

    public void display() {
        System.out.print(this.age);
    }
}

class Main {

    public static void main (String arg[]) {
        Student obj = new Student();
        obj.display();
    }
}
```



1. `this` is placed automatically in front of the instance variables
2. It indicates the reference of the calling object
3. You can also explicitly place this in front of the members of the class

The keyword `this` needs to be **placed explicitly in methods and constructors, if local variables and/ or parameters have the same name as the instance members.**

```
class Student {

 private int age;

 public Student (int age) {
    this.age = age;
 }
}
```

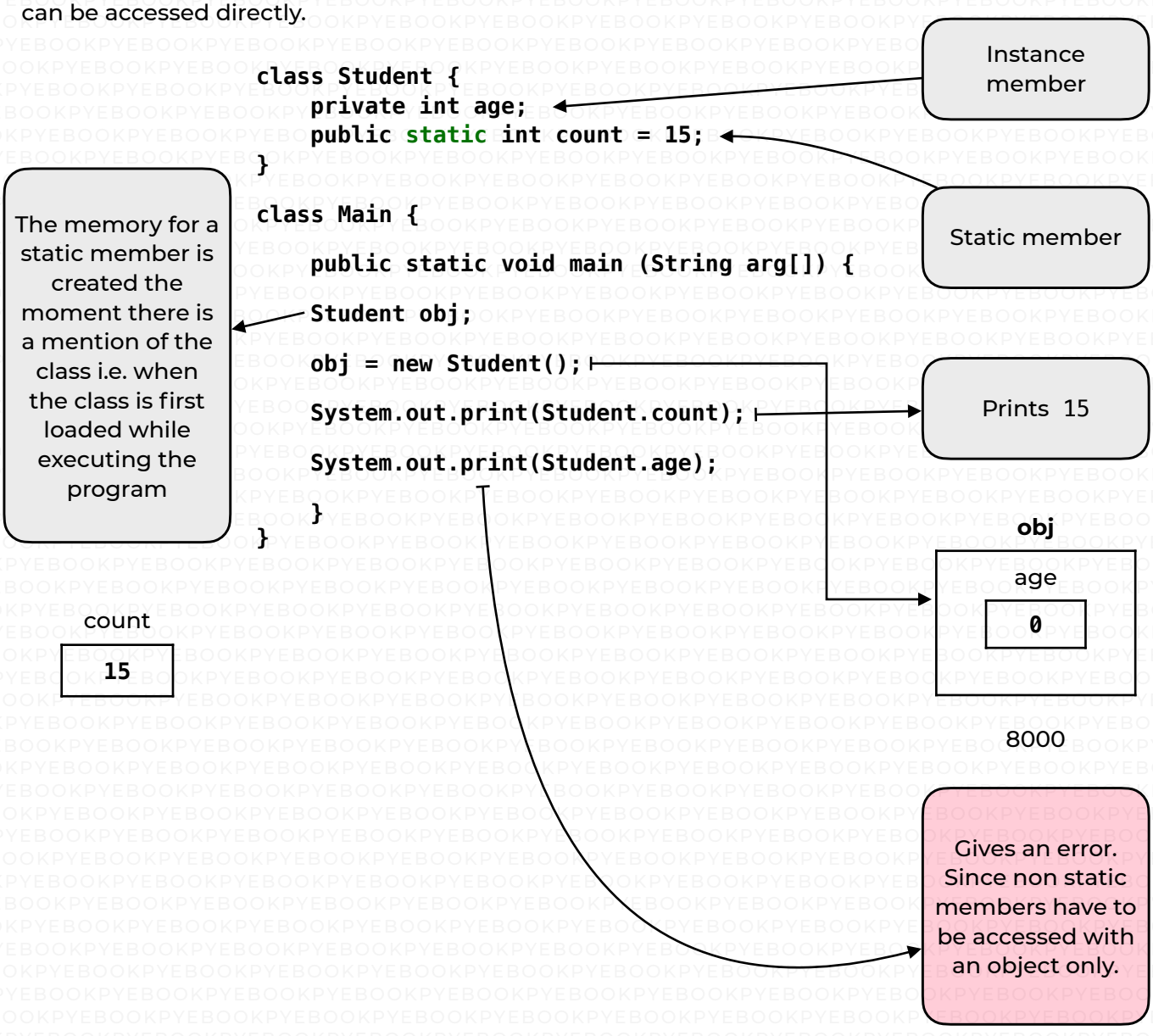
If there is an instance variable and a local variable with the same name, then the preference is given to the local variable and hence `this.` should be placed to refer to the instance variable

8. Static Members and Methods

STATIC DATA MEMBERS

CONCEPT

1. Instance members belong to the object and hence the number of copies depends on the number of instances created.
2. Static data members, only 1 copy is created.
3. All the instances share the same copy of the variable.
4. If possible(i.e. if it is public) it needs to be accessed with the class name. Within the same class it can be accessed directly.



STATIC METHODS

CONCEPT

1. Usually used to work with static members
2. Since it belongs to the class, it does not have the `this` keyword inside it
3. Therefore non static or instance members can't be accessed **directly** inside it
4. Static members can be accessed in any method - static or non-static

EXAMPLE

```
class Student {

    private int age;
    public static int count = 15;

    public void display(){
        System.out.print (this.age);
        System.out.println(count);
    }

    public static void inc() {
        System.out.print(age);
        Student obj = new Student();
        obj.age = 20;
        count++;
    }
}

class Main {

    public static void main (String arg[]) {

    Student.inc();

    }
}
```

There is an object referencing age and hence this is allowed

Static members can be accessed inside non static methods

Since there is no `this` inside a static method, the non static members cannot be accessed directly

Accessing a static method outside the class with the class name

9. Pass by Value & Pass by Reference

CONCEPT & EXAMPLE

```
class ArrayVariableExample{

    public static void changeArrayVariable(int
arr[], int val){

        arr[0]++;

        val +=1;

    }

    public static void main(String[] args) {
        int []vals = {10,40,50,25};

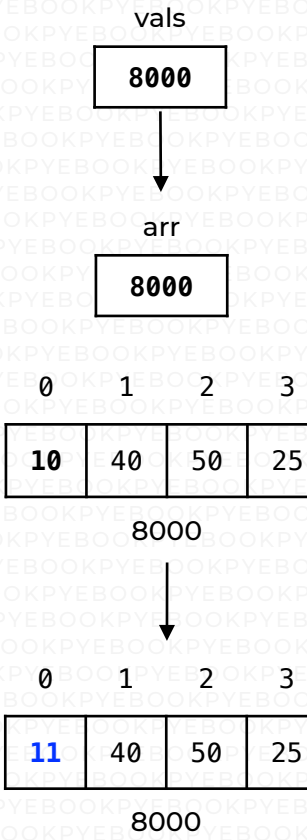
        int var = 20;

        changeArrayVariable(vals, val);

        System.out.println(vals[0]+" "+var);

    }
}
```

Since arr and vals both point to the same array, the change made will be reflected in the original array





val is storing the value of var.Hence the changes are reflected only in val and **not in** var

The output is 11 20

The reference of vals is copied whereas the value of val is copied. The array is passed as reference whereas primitive types are **passed by value**

10. Working With References & Aliasing

CONCEPT & EXAMPLE

```
class SomeNumber{
    private int x;

    public SomeNumber(int x){
        this.x = x;
    }

    public void incBy1(){
        this.x+=1;
    }

    public void display(){
        System.out.println(x);
    }
}

public class Main
{
    public static void main(String[] args) {

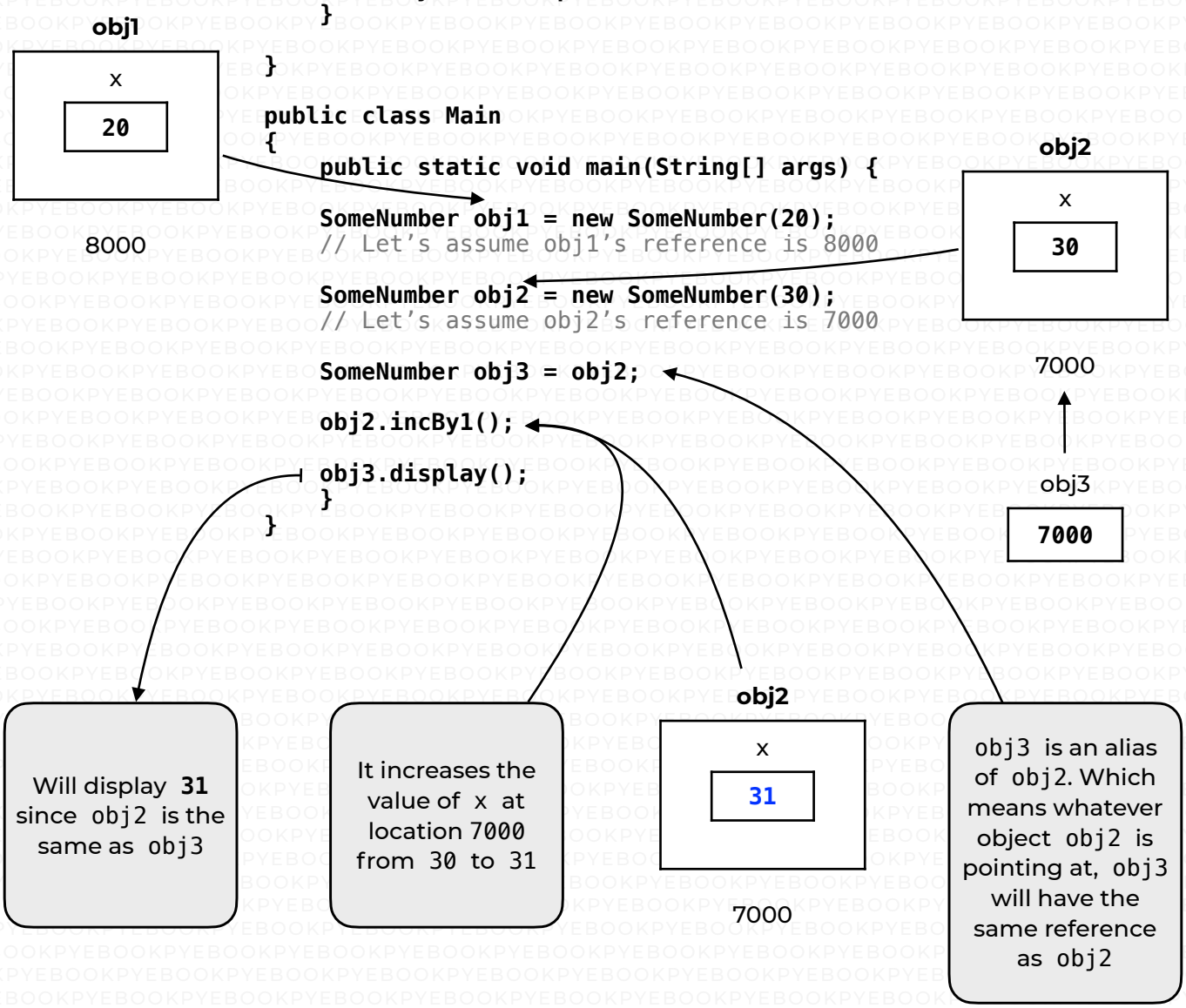
        SomeNumber obj1 = new SomeNumber(20);
        // Let's assume obj1's reference is 8000

        SomeNumber obj2 = new SomeNumber(30);
        // Let's assume obj2's reference is 7000

        SomeNumber obj3 = obj2;

        obj2.incBy1();

        obj3.display();
    }
}
```



11. Math Class

All members and methods of the **Math class are static** and hence referenced by the name of the class.

1. public final static PI

CONCEPT & EXAMPLE

`final static` data member of the Math class. The value is value of the mathematical notation π.

Example:

```
int radius = 4;
double area = Math.PI * radius *
radius // Calculates area of circle
```

2. public static int abs(int x)

CONCEPT & EXAMPLE

Static method which returns the absolute value(positive value) of an `integer`.

Example:

```
int y = -4;
int ans = Math.abs(y);
```

ans will have the value:

```
4
```

3. public static double abs(double x)

CONCEPT & EXAMPLE

Static method which returns the absolute value(positive value) of a `double`.

Example:

```
double y = -5.2;
double ans = Math.abs(y);
```

ans will have the value:

```
5.2
```

4. public static double pow(double a, double b)

CONCEPT & EXAMPLE

Returns a^b.

Example:

```
int y = 2, z = 3;
double ans = Math.pow(y, z);
```

ans will have the value:

```
8.0
```

5. public static double sqrt(double val)

CONCEPT & EXAMPLE

Returns the positive square root of the number.

Example:

```
int y = 4;
double ans = Math.sqrt(y);
```

Output:

```
2.0
```

6. public static double random()

CONCEPT & EXAMPLE

Returns a random **between 0.0 (included) and 1.0 (not inclusive)**.

Example:

```
double val = Math.random();
```

`val` can be:

```
0.12
```

NOTE

`random()` can be used to generate a random integer between `n1` and `n2` (where `n1 < n2`)

```
int rand = (int)(Math.random()*(n2-n1+1))
+ n1;
```

12. String Class

A `String` (a secondary data type) is an object of the String class which denotes **a collection of one or more characters enclosed in quotation marks.**

DIFFERENT WAYS OF CREATING STRINGS

CONCEPT

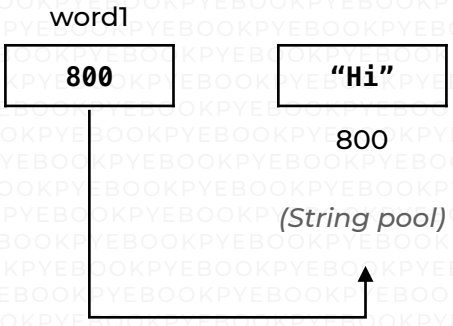
Using the quotes	Using the constructor
`String word1 = "Hello";`	`String word2 = new String("Hello");`
Strings are created in the String pool	Strings are created outside the String pool
Inside the string pool, no duplicates can exist	Outside the string pool, duplicates can exist

NOTE

The name of the `String` **is a reference** of where the String is stored.

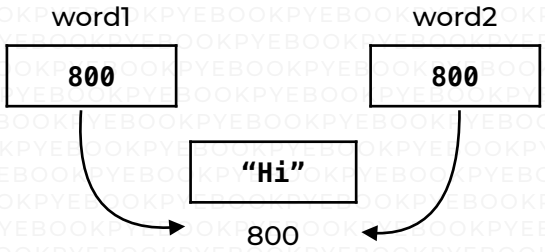
EXAMPLES

```
String word1 = "Hi";
// word1 is reference to "Hi"
```



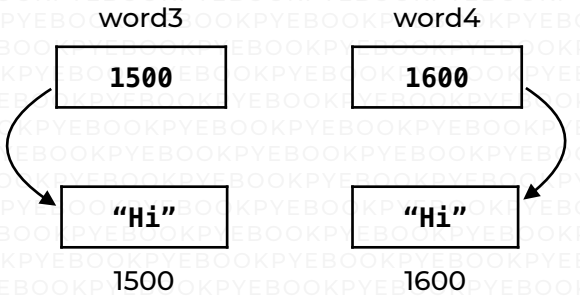
(String pool)

```
String word2 = "Hi";
// word2 references to the same "Hi"
since duplicates can't exist in the pool
```



(String pool)

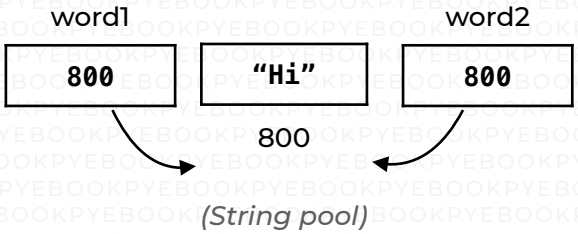
```
String word3 = new String("Hi");
String word4 = new String("Hi");
// Both word3 and word4 are outside the
pool and are at different locations
```



(Outside String pool) (Outside String pool)

For the above set of Strings created (the memory allocation is shown above).

```
boolean ans1 = word1 == word2;
```



(String pool)

The value of `ans1` here will be:

```
true
```

```
boolean ans2 = word1 == word3;
```

The value of ans2 here will be:

```
false
```

```
boolean ans3 = word3 == word4;
```

The value of ans3 here will be:

```
false
```

NOTE

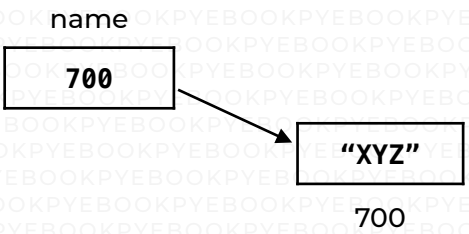
Here the references are being compared and not the contents.

STRING IMMUTABILITY

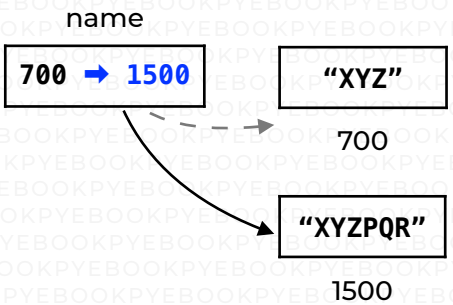
CONCEPT & EXAMPLE

Once a string is created at a particular location, **at that location the contents of the String can't be changed.**

```
String name = "XYZ";
```




```
name = name + "PQR";
```



What you saw was when you made the change to name, it actually created a new memory location with the new String and the reference of the name changed.

NOTE

Whenever Strings are modified, a new String is created. When Strings are passed to methods and the methods modify the String, then the String in the calling function remains unchanged.

STRING METHODS

1. public int length()

CONCEPT & EXAMPLE

Returns the length of the String object.

Example:

```
String word = "APCSA";
int len = word.length();
```

len will have the value :

```
5
```

2. public String substring(int startPos)

CONCEPT & EXAMPLE

Returns the part of the string starting from `startPos` till the end. Remember positioning begins from 0.

Example:

```
String word = "APCSA";
String part = word.substring(2);
```

part will have the value:

```
CSA
```

3. public String substring(int startPos, int endPos)

CONCEPT

Returns the part of the string starting from `startPos` **till the endPos not including endPos**.

EXAMPLE

Example 1:

```
String word = "APCSA";
String part = word.substring(2,4);
```

part will have the value:

CS

Example 2:

```
String word = "APCSA";
String part = word.substring(4,5);
```

part will have the value:

A

Example 3:

```
String word = "APCSA";
String part = word.substring(5,6);
```

The output will be:

StringIndexOutOfBoundsException

4. public boolean equals(Object other)

CONCEPT & EXAMPLE

Returns true if the current string is exactly the same as the other string otherwise false.

Example:

```
String word1 = "AP";
String word2 = new String("AP");
String word3 = "ap";
boolean ans = word1.equals(word2);
boolean ans2 = word1.equals(word3);
System.out.print(ans + " " + ans2);
```

The output will be:

true false

5. public int indexOf(String part)

CONCEPT & EXAMPLE

Returns the position of the 1st occurrence where the part is in the current string. If the part doesn't exist then it's –1.

Example:

```
String word = "APCSA";
int val = word.indexOf("A");
int val2 = word.indexOf("PC");
int val3 =word.indexOf("PCT");
System.out.print(val + " " + val2 +
" " + val3);
```

The output will be:

0 1 -1

6. public int compareTo(String other)

CONCEPT & EXAMPLE

Returns

1. **a value < 0 if this is less than other**
2. **zero if this is equal to other**
3. **a value >0 if this is greater than other**

Example:

```
String word1 = "APCSA";
String word2 = "CS";
int val = word1.compareTo(word2);
// -ve value indicating word1 comes
// alphabetically ahead of word2

int val2 = word2.compareTo(word1);
// +ve indicating word2 comes
alphabetically after word1

String word3 = "CS";
int v = word2.compareTo(word3);
// gives 0
```

SOME IMPORTANT STRING ALGORITHMS

A. STRING COMPARISON

CONCEPT

If there are two Strings, in order to check whether they have the same contents or not use either **the equals** method or **the compareTo** method.

ALGORITHM

```
String word1 = "XYZ";
String word2 = "PQR";
if(word1.equals(word2))
   System.out.println("Words are same");
else
   System.out.println("Words not same");
```

NOTE

A very common mistake is to write the `if` condition as:

```
if(word1 == word2)
```

This **does not** compare the contents but instead compares the references.

B. ACCESSING EACH LETTER OF A STRING

ALGORITHM

We use substring to do the same.

```
String word = "APCSA";
for(int i = 0 ; i <word.length(); i++)
   System.out.println(word.substring(i,
i+1));
```

C. CHECK PRESENCE OF A STRING/ CHARACTER IN ANOTHER STRING

CONCEPT

Use `indexOf` to check whether a String or a character is present in another String or not. If it's not present indexOf will return `-1`

ALGORITHM

```
String word1 = "XYZ";
String word2 = "PQR";
if(word1.indexOf(word2) != -1)
   System.out.println("word2 is present
in word1 ");
else
   System.out.println("word2 is not
present in word1);
```

D. CREATE A NEW STRING FROM AN EXISTING STRING

CONCEPT

If the requirement is to create a new String from another String, then start out with an empty String and keep appending to it. The code below creates a new string without the letter **a** in it.

ALGORITHM

```
String word = "data";
String newword = "";
for(int i = 0 ; i <word.length(); I++) {
   if(!word.substring(i,
i+1).equals("a")) {
      newword += word.substring(i, i+1);
   }
}
```

E. REPLACING THE OCCURRENCE OF THE OLD STRING WITH THE NEW STRING.

CONCEPT

Logic is as follows:

1. Store the original string in an another String.
2. Find the occurrence of old in the other String.
3. If present change the other string with the new string and go back to step 2. When there are no more occurrences left, then stop the process.

ALGORITHM

```
String input;
// Some input
String old;
// word to be replaced
String newWord;
// Word to be replaced with

String updated = input;
int found = updated.indexOf(old);
// find first location of old
while (found != -1) {
   // loop while old is still in the
String
   // replace old with newWord
   updated = updated.substring(0, found)
+ newWord + updated.substring(found +
old.length());
   found = updated.indexOf(old); // find
next occurrence;
}
System.out.println(updated);
```

Table of Contents

1. Basics of ArrayList & drawbacks of arrays
2. Working with array of objects
3. Working with ArrayList of objects
4. Wrapper classes
5. Integer class members and methods
6. Double class methods
7. Auto boxing and Auto unboxing
8. Standard ArrayList algorithms

1. Basics of ArrayList & Drawbacks of Arrays

CONCEPT

For an array,

1. Size needs to be specified while array creation or the values need to be given directly.
2. Once created, the size cannot change.
3. If you need to modify the array, the only way is to create a new array with the modified size and copy the values from the original to the new array if needed.

`ArrayList` is a **dynamic data structure** which can alter its size on the fly and does not have to have a specific size mentioned at the time of creation. It is part of the `java.util` package.

ARRAYLIST CREATION

```
ArrayList<E> list = new ArrayList<E>();
```

NOTE

E mentioned above can **only be a secondary datatype** like `String`, or any other user defined class. It cannot be primitive data types like `int`, `boolean` and `double`.

EXAMPLE

Consider an ArrayList of Strings declared below.

```
ArrayList<String> names = new
ArrayList<String>();
```

For the below set of methods, `names` is the `ArrayList` which is used all throughout below

ARRAYLIST METHODS

`1. public boolean add(E obj)`

CONCEPT & EXAMPLE

Adds the **object to the end of the list**

Code snippet:

```
names.add("XYZ");
names.add("PQR");
```

Contents of **`names`** after the above code:

```
["XYZ", "PQR"]
```

`2. public int size()`

CONCEPT & EXAMPLE

Returns the **number of elements** in the list.

Code snippet:

```
int val = names.size();
```

Contents of **`val`** after the above code:

```
2
```

`3. public void add(int index, E obj)`

CONCEPT & EXAMPLE

Inserts obj at position index (`0 <= index <= size`), moving elements at position `index` and higher to the right (adds 1 to their indices) and adds 1 to size.

`names` list so far is:

```
["XYZ", "PQR"]
```

Now when we do,

```
names.add(1, "LMN");
```

Contents of **`names`** after the above code:

```
["XYZ", "LMN", "PQR"]
```

3. Working With ArrayList of Objects

For the same `class Student` above.

```
class Student {
    private String name;
    private int age;

    public Student(String name, int age){
        this.name = name;
        this.age = age;
    }

    public int getAge(){
        return age;
    }

    public boolean isAdult(){
        return age > 18;
    }
}
```

The following piece of code creates an `ArrayList` of Student objects and performs operations with it.

CREATING AN ARRAYLIST AND ADDING OBJECTS TO IT

CONCEPT

Code snippet:

```
ArrayList<Student> students = new
ArrayList<Student>();
students.add(new Student("LMN",15));
students.add(new Student("LOP",19));
```

TRAVERSING THE ARRAYLIST USING A NORMAL FOR LOOP

CONCEPT

The code below counts the number of students that are adults.

Code snippet:

```
int count = 0;
for(int i = 0 ; i<students.size(); i++)
{
    if(students.get(i).isAdult()){
        count++;
    }
}
```

NOTE

Common mistakes while traversing an `ArrayList` is to use the `length` instead of `size` and is to do `students[i]` inside the loop to access a Student object.

TRAVERSING THE ARRAYLIST USING THE FOR EACH LOOP

CONCEPT

The for each loop can be used to traverse both arrays and ArrayLists.

The code for counting the number of adults can be seen below.

Code snippet:

```
int count = 0;
for(Student s: students){
    if(s.isAdult()){
        count++;
    }
}
```

4. Wrapper Classes

CONCEPT

Java created wrapper classes for each of the primitive data types. With ArrayLists, you can now work with these wrapper classes if you intend to work with `integers` or `doubles`.

Primitive Type	Wrapper Class
`int`	`Integer`
`double`	`Double`

5. Integer Class Members and Methods

1. Integer.MIN_VALUE

The minimum value represented by an int or Integer. Its a **static field** of the `Integer class`

Example:
It can be used to find the maximum value in an array.

Part of the code snippet:

```
int max = Integer.MIN_VALUE;
//Rest of the code to find maximum
in a 1D Array
```

2. Integer.MAX_VALUE

The maximum value represented by an int or Integer. It's a **static field** of the `Integer class.`

Example:
It can be used to find the minimum value in an array.

Part of the code snippet:

```
int max = Integer.MAX_VALUE;
//Rest of the code to find minimum
in a 1D Array
```

3. Integer(int value)

This is the **constructor of the `Integer class`** which creates an **Integer object using an `int`.**

Example:

```
Integer obj = new Integer(15);
//obj can be added in a list of
Integer objects
```

4. public int intValue()

`intValue` is a **non-static method** of the Integer class which gives the `int` value of the corresponding object.

Example:

```
ArrayList<Integer> list = new
ArrayList<Integer>;
list.add(new Integer(20));
Integer ob = list.get(0);
int val = ob.intValue();
```

6. Double Class Methods

1. Double(double value)

This is the **constructor of the `Double class`** which creates a **Double object using a `double`.**

Example:

```
Double obj = new Double(10.2);
//obj can be added in a list of
Double objects
```

2.public double doubleValue()

`doubleValue` is a **non-static method** of the Double class which gives the **`double`** value of the corresponding object.

Example:

```
Double v = new Double(10.3);
double m = v.doubleValue();
```

7. Auto Boxing and Auto Unboxing

Java allows to convert from secondary data types to primitive types and the other way round automatically.

Code:

```
Integer val = 20; //Auto boxing
//Implicitly calls the constructor

int value = val; //Auto unboxing
//Implicitly calls the intValue
```